

Описание классов Pioneer-sdk2

Описание класса Pioneer

Создание экземпляра класса Pioneer

`drone_1 = Pioneer()` - создает переменную `drone_1` которой присваивается объект класса `Pioneer()`.

```
from pioneer_sdk2 import Pioneer # импортируем класс Pioneer из библиотеки
pioneer_sdk2drone_1 = Pioneer()   # создаем экземпляр класса Pioneer,
устанавливаем соединение
```

Описание класса Camera

Класс `Camera` позволяет получить изображение с камеры квадрокоптера и при необходимости обработать его.

Создание экземпляра класса Camera

`Camera(camera_type=CameraType.MAIN)` - основная камера, по умолчанию применяется этот тип камеры.

`Camera(camera_type=CameraType.OPT)` - камера оптической навигации, расположена на плате-адаптере. Не работает на модуле `Radxa Zero`.

Получить кадр и выключить видеопоток

`get_cv_frame(timeout)` - возвращает следующий доступный кадр из очереди в формате BGR.

`timeout` - время ожидания кадра в секундах

`stop()` - останавливает передачу кадров.

```
from pioneer_sdk2 import Camera # импортируем класс Camera из библиотеки pioneer_sdk2
import cv2                      # библиотека cv2 содержит функции для работы с изображениями
```

```
camera_drone_1 = Camera()      # создаем экземпляр класса Camera
```

```
while True:                    # запускаем бесконечный цикл
```

```
    frame = camera_drone_1.get_cv_frame() # сохраняем изображение в переменную frame
```

```
    if frame is not None:       # проверяем, что изображение получено
```

```
        cv2.imshow("video", frame) # выводим полученное изображение frame в окне с
названием "video"
```

Описание класса ImageViewer

Класс для трансляции нумпу-кадров по RTSP через `GStreamer`, доступен только при использовании драйвера камеры `gststreamer`.

Создание экземпляра класса ImageViewer

```
from pioneer_sdk2 import ImageViewer # импортируем класс ImageViewer из
```

```

библиотеки pioneer_sdk2
video_drone_1 = ImageViewer()          # создаем экземпляр класса ImageViewer,
камера модуля esp32

```

Включить и выключить видеотрансляцию

imshow(name, frame, fps=30) - запускает видеотрансляцию в браузер.

close() - уничтожает все gstreamer пайплайны.

```

from pioneer_sdk2 import ImageViewer, Camera # импортируем классы из библиотеки
pioneer_sdk2
import time                                   # библиотека time содержит функции для работы со временем

video_drone_1 = ImageViewer()                # создаем экземпляр класса ImageViewer
camera_drone_1 = Camera()                    # создаем экземпляр класса Camera

my_time = time.time()                        # объявляем переменную my_time, присваиваем текущее
время (в секундах с 1970 года)
cycle_time = 30                              # объявляем переменную cycle_time, присваиваем желаемую
длительность цикла

while time.time() - my_time < cycle_time:    # запускаем цикл, код будет повторяться, пока
условие верно
    frame = camera_drone_1.get_cv_frame()    # получаем кадр и присваиваем данные
переменной frame
    video_drone_1.imshow("video", frame, fps=30) # запускаем трансляцию, содержит
аргументы:
                                                # video - название трансляции
                                                # frame - переменная с ранее полученным кадром
                                                # fps=30 - количество кадров в секунду при передаче видео

                                                # трансляция выполняется по адресу: 10.42.0.1:8889/video
                                                # ip - по умолчанию 10.42.0.1 или скопировать из раздела хот-
спот на компьютере

                                                # 8889 - порт трансляции
                                                # name - название трансляции

                                                # при подключения квадрокоптера к вашей сети:
                                                # трансляция выполняется по адресу: ваш ip:8889/video

video_drone_1.close()                        # останавливаем видеопоток

```

Описание класса ServoCamera

Класс для управления сервоприводом камеры, позволяющий устанавливать угол поворота.

Создание экземпляра класса ServoCamera

```

from pioneer_sdk2 import ServoCamera # импортируем класс ServoCamera из
библиотеки pioneer_sdk2
servo_drone_1 = ServoCamera()          # создаем экземпляр класса ServoCamera,
проверяет поддержку сервомотора

```

Установить угол сервопривода камеры

`set_angle(angle)` - устанавливает угол поворота сервопривода камеры, диапазон от -90 до 30 градусов.

- Возвращает True, если выполнено, иначе False.
- Вызывает Exception ValueError, если угол выходит за пределы допустимого диапазона (от -90 до 30 градусов).

```
from pioneer_sdk2 import ServoCamera # импортируем класс ServoCamera из библиотеки
pioneer_sdk2
import time # библиотека time содержит функции для работы со временем
```

```
servo_drone_1 = ServoCamera() # создаем экземпляр класса ServoCamera, проверяет
поддержку сервомотора
```

```
servo_drone_1.set_angle(25) # устанавливаем угол камеры на 25 градусов
time.sleep(3) # ставим паузу на 3 секунды
servo_drone_1.set_angle(-25) # устанавливаем угол камеры на -25 градусов
```

Управление полетом

`arm(timeout=5, retries=0)`

Описание метода:

Запуск моторов квадрокоптера

Аргументы метода:

- `timeout` - задержка перед запуском моторов
- `retries` - количество повторных попыток запуска моторов

Тип данных:

bool

Возвращает:

True, если команда успешно выполнена, иначе False

Примечание:

Ожидает события `ENGINES_STARTED` при параметре `Pioneer.wait_callback=True`

Включить и выключить двигатели

disarm()

Описание метода:

Выключение моторов квадрокоптера

Тип данных:

bool

Возвращает:

True, если команда успешно выполнена, иначе False

```
from pioneer_sdk2 import Pioneer # импортируем класс Pioneer из библиотеки pioneer_sdk2
import time                       # библиотека time содержит функции для работы со временем

drone_1 = Pioneer()              # создаем экземпляр класса Pioneer, устанавливаем соединение

drone_1.arm(timeout=5, retries=0) # включаем двигатели, содержит аргументы:
                                   # timeout(5) - задержка в секундах перед запуском двигателей
                                   # retries(0) - количество повторных попыток запуска двигателей

time.sleep(3)                    # ставим паузу на 3 секунды
drone_1.disarm()                  # выключаем двигатели
drone_1.close_connection()       # закрываем соединение
```

takeoff()

Описание метода:

Запуск процесса взлёта

Тип данных:

bool

Возвращает:

True, если команда успешно выполнена, иначе False

Примечание:

Ожидает события TAKEOFF_COMPLETE при параметре wait_callback=True

Взлет и посадка

land()

Описание метода:

Запуск процесса посадки

Тип данных:

bool

Возвращает:

True, если команда успешно выполнена, иначе False

Примечание:

Ожидает события COPTER_LANDED при параметре wait_callback=True

```
from pioneer_sdk2 import Pioneer # импортируем класс Pioneer из библиотеки pioneer_sdk2
import time                       # библиотека time содержит функции для работы со временем
```

```
drone_1 = Pioneer()              # создаем экземпляр класса Pioneer, устанавливаем соединение
```

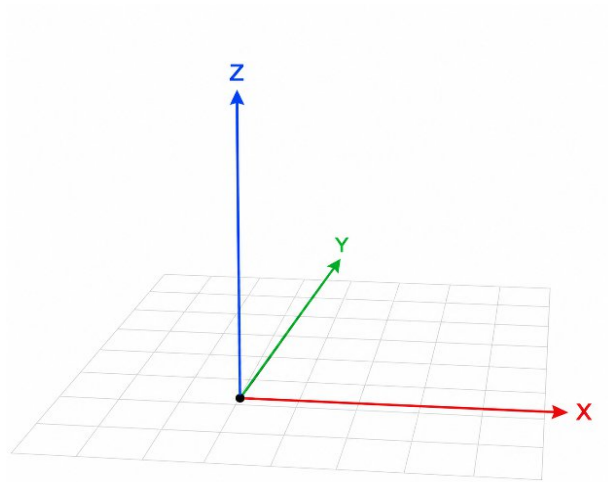
```
try:                             # конструкция try-except, основной код находится внутри блока try
    drone_1.arm()                 # включаем двигатели
    drone_1.takeoff()             # взлетаем
    time.sleep(3)                # ставим паузу на 3 секунды
    drone_1.land()                # садимся, двигатели выключатся автоматически
    drone_1.close_connection()    # закрываем соединение
```

```
except:                          # конструкция try-except, в случае ошибки будет выполнен блок except
    drone_1.land()
    drone_1.close_connection()
```

Полет в координаты (локальные)

go_to_local_point(x, y, z, yaw, time) - полет в заданную координату, на основе локальной системы координат.

При запуске моторов квадрокоптер запоминает точку старта как (0, 0, 0). При этом ось y направлена вперед (по направлению квадрокоптера), ось x - вправо. Параметр z отвечает за высоту полёта. Значения (x, y, z) в метрах задают координаты точки, в которую полетит квадрокоптер, относительно точки старта.



`go_to_local_point(x, y, z, yaw, time=0)`

Описание метода:

Перемещает квадрокоптер в указанную координату относительно локальной системы координат

Аргументы метода:

- x (float) - Координата X в метрах
- y (float) - Координата Y в метрах
- z (float) - Координата Z в метрах
- yaw (float) - Угол рысканья в градусах
- time (int) - Желательное время достижения целевой точки

```
from pioneer_sdk2 import Pioneer # импортируем класс Pioneer из библиотеки pioneer_sdk2
import time                       # библиотека time содержит функции для работы со временем
```

```
drone_1 = Pioneer()              # создаем экземпляр класса Pioneer, устанавливаем соединение
```

```
try:                             # конструкция try-исключение, основной код находится внутри блока try
    drone_1.arm()                 # включаем двигатели
    drone_1.takeoff()             # взлетаем
    time.sleep(3)                # ставим паузу на 3 секунды
    drone_1.go_to_local_point(0, 0, 1.5, 0, 0) # вызываем метод, содержит аргументы:
                                     # x(0) - координата по оси "x" (в метрах)
                                     # y(0) - координата по оси "y" (в метрах)
                                     # z(1.5) - координата по оси "z" (в метрах)
                                     # yaw(0) - поворот по курсу (в радианах)
                                     # time(0) - время за которое требуется достигнуть координату
```

```
time.sleep(5)                    # ставим паузу на 5 секунд
drone_1.land()                   # садимся, двигатели выключаются автоматически
drone_1.close_connection()       # закрываем соединение
```

```
except:                          # конструкция try-исключение, в случае ошибки будет выполнен блок except
```

```
drone_1.land()  
drone_1.close_connection()
```

Узнать локальные координаты

`get_local_position_lps()` - позволяет узнать локальные координаты. None, если ошибка.

```
from pioneer_sdk2 import Pioneer # импортируем класс Pioneer из библиотеки pioneer_sdk2  
import time # библиотека time содержит функции для работы со временем
```

```
drone_1 = Pioneer() # создаем экземпляр класса Pioneer, устанавливаем соединение
```

```
while True: # запускаем бесконечный цикл  
    print(drone_1.get_local_position_lps(True)) # вызываем метод  
    time.sleep(1) # ставим паузу на 1 секунду
```

```
drone_1.close_connection() # закрываем соединение
```

Узнать высоту (дальномер)

`get_dist_sensor_data()` - позволяет узнать высоту с дальномера. None, если ошибка.

```
from pioneer_sdk2 import Pioneer # импортируем класс Pioneer из библиотеки pioneer_sdk2  
import time # библиотека time содержит функции для работы со временем
```

```
drone_1 = Pioneer() # создаем экземпляр класса Pioneer, устанавливаем соединение
```

```
while True: # запускаем бесконечный цикл  
    print(drone_1.get_dist_sensor_data()) # вызываем метод  
    time.sleep(1) # ставим паузу на 1 секунду
```

```
drone_1.close_connection() # закрываем соединение
```

Пример кода для полета по координатам:

```
import time  
from pioneer_sdk2 import Pioneer
```

```
mini = Pioneer()
```

```
mini.arm()  
mini.takeoff()  
time.sleep(3)
```

```
mini.go_to_local_point(0, 1, 2, 0, 0)  
while not mini.point_reached():  
    time.sleep(0.05)  
mini.go_to_local_point(1, 1, 2, 0, 0)  
while not mini.point_reached():  
    time.sleep(0.05)
```

```
mini.go_to_local_point(1, 0, 2, 0, 0)
while not mini.point_reached():
    time.sleep(0.05)
mini.go_to_local_point(0, 1, 2, 0, 0)
while not mini.point_reached():
    time.sleep(0.05)
```

```
mini.land()
mini.disarm()
```

Пример кода для получения видеопотока с камеры:

```
import time
from pioneer_sdk2 import Pioneer, Camera, ImageViewer, ServoCamera

mini = Pioneer()
camera = Camera()
video = ImageViewer()
servo_drone_1 = ServoCamera()

time.sleep(3)

servo_drone_1.set_angle(30)

while True:                                # запускаем бесконечный цикл
    frame = camera.get_cv_frame(timeout=5.0) # сохраняем изображение в переменную frame
                                              # timeout=5.0 - время ожидания кадра в секундах

    if frame is not None:                   # проверяем, что изображение получено
        video.imshow("video", frame)       # выводим полученное изображение frame в
окне с названием "video"
```